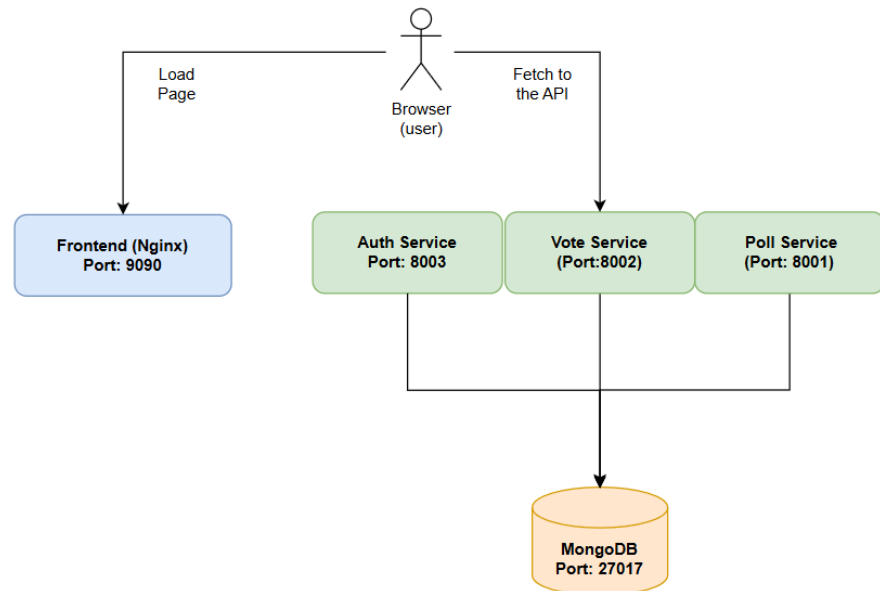


Poll Vote App (Assignment 'build something')

PollVoteApp is an application created to imitate the 2026 Swedish general election that happened during the first weeks of September. It enables users to create and vote for polls in real time with its microservice architecture, containerized with Docker and orchestrated by Kubernetes.

❖ Architecture structure

The application is divided into the following independent components (drawn with draw.io):



❖ Auth Service

(`Containers/AuthService`):

Handles the registration and the login of the user. Saves the cyphered passwords with Bcrypt on MongoDB and generates the JSON Web Tokens. Thanks to the stateless authentication it remains scalable horizontally. The port for this microservice is the 8003.

❖ Vote Service (`Containers/VoteService`):

Handles the polls votes, also verifies on MongoDB if the user has already voted to that specific poll. The port for this microservice is the 8002.

❖ Frontend (`Containers/Frontend`):

Serviced static application serviced with Nginx (internal port 80 exposed on 9090). It's composed of the login, registration, creation of the polls and voting parts. Also shows the status of the different polls votes to the user.

❖ Database ('MongoDB'):

The application uses NoSQL database on the port 27017 with persistent volume claim, ('mongo-pvc') to grant the persistence of the data even after the restart of the container/pod. I've decided to use MongoDB because the voting structure can be represented with a hierarchical structure (a poll contains a list of options and the respective votes and a list of the users voting). MongoDB is also easily updatable with operators such as '\$inc' for the votes or '\$push' to add another user to the voting list.

■ Benefits of the Architecture Design

- **Separation of Concerns:** The application is decoupled into distinct, independent microservices, each managing their own bounded domain and lifecycle. With this type of architecture, the failure or change in one service does not take down the others.

- **Independent Horizontal Scalability:** Because the FastAPI microservices are completely stateless, we can independently scale only the service experiencing high load (e.g., scaling VoteService replicas during a surge of votes) without wasting compute resources on idle services.
- **Domain-Technology Alignment:** FastAPI provides high-throughput asynchronous execution (ASGI/Uvicorn); MongoDB's NoSQL document model naturally reflects the hierarchical structure of polls, nested options, vote tallies, and voter arrays without expensive relational JOIN operations.
- **Resilience and Self-Healing:** Kubernetes provides automated pod recovery, container health monitoring, and rolling updates with zero downtime.

❖ Patterns used in the application

- ❖ **Microservices Architecture:** The system is decomposed into small, single-responsibility services organized around specific business domains.
- ❖ **Stateless Services:** All application containers maintain no session state in memory or on local container storage. Requests can hit any replica interchangeably, allowing effortless horizontal autoscaling (kubectl scale).
- ❖ **Containerization & Orchestration:** The application relies entirely on Docker (for packaging dependencies and runtimes) and Kubernetes (for automated deployment, scaling, and management). This isolates the application from the underlying infrastructure, utilizing standardized .yaml manifests for Deployments, Services, and PersistentVolumeClaims.
- ❖ **Decoupled Client-server:** The frontend consists of static HTML and JavaScript that communicates asynchronously with the backend services strictly over standard REST APIs via HTTP and JSON.
- ❖ **Persistent Volume Abstraction:** MongoDB state is decoupled from the ephemeral container lifecycle using a Kubernetes Persistent Volume Claim (mongo-pvc), ensuring data persistence across pod restarts and rescheduling.
- ❖ **Dependency Injection (DI):** Utilized in FastAPI through Depends the route handlers stay clean by delegating token parsing, decoding and authentication logic to reusable dependency providers.

■ Security

Implemented Security Measures:

- **Password Hashing with Bcrypt:** User credentials are never stored in plaintext: passwords are salted and hashed using Bcrypt, mitigating rainbow table attacks.
- **Stateless JWT Authentication (HS256):** Cryptographically signed JSON Web Tokens with a 24-hour expiration to protect against replay attacks. Mutating endpoints are secured via FastAPI's dependency injection.
- **Database Network Isolation:** MongoDB is exposed strictly via an internal Kubernetes ClusterIP Service (port 27017). It is completely inaccessible from outside the cluster network, preventing direct external attacks.

- **Input Validation & Injection Prevention:** Pydantic models enforce strict request typing and payload validation. In addition, MongoDB ObjectId conversions are wrapped in try-except blocks to eliminate unhandled exceptions and mitigate NoSQL injection vulnerabilities.

Future Security Enhancements:

- **Centralized Secret Management:** Migrate the JWT_SECRET from plaintext environment variables in manifests to encrypted Kubernetes Secrets (KMS-backed) or an external vault like HashiCorp Vault.
- **Transport Layer Security (TLS/HTTPS):** Deploy an Ingress Controller with automated TLS certificate management via Cert-Manager to encrypt all client-to-cluster traffic.
- **Rate Limiting & Anti-Brute-Force:** Add rate-limiting middleware to /login and /register to prevent brute-force credential stuffing and Denial-of-Service (DoS) attacks.

■ Scalability notes

As asked by the requirements for the project all the microservices can scale horizontally and independently without causing the interruption of the service. In a possible scenario in which the application is open to the public and used by many users then there would be many thousands of votes being sent in a short time span. This would cause more writing on the database than reading, but thanks to the containerization and to FastAPI the application can scale without trouble. Kubernetes automatically distributes incoming requests across all replicas of vote-deployment via the Service's internal load balancing, regardless of the Service type used for external exposure. Another solution would be to use the 'Sharding' technique on the MongoDB to create replica sets.

e.g. 'kubectl scale deployment vote-deployment --replicas=3'

❖ Known Limitations

The known limitations of the current application are:

- ❖ **Single instance database:** even though there is PVC on MongoDB the absence of a cluster ReplicaSet or a Sharded means that the database doesn't offer high availability nor horizontal database scalability.
- ❖ **Parallel writing on the Database:** In the project a message queue is not implemented, so extremely high concurrent write frequency on the same specific poll document could create contention, even though MongoDB's atomic \$inc operator prevents data corruption.
- ❖ **Secret management:** the JWT ciphered is defined clearly in the deployment.yaml manifest, it should be managed with Kubernetes secrets or other services.

■ Git

If you need the code online this is the link to the repository <https://github.com/Cyb-CL/PollVoteApp>